StatementIQ

Enterprise Bank Statement Extraction, Dual OCR/Digital Pipeline, Non-LLM Transaction Classification & Financial Reconciliation System

Author / Candidate: Riti Shah	**Target Environment:** Production / Cloud / Local
Repository: github.com/ritishah15/statementiq	**Frontend:** React 18, Vite, Tailwind CSS
ML Architecture: TF-IDF + Multinomial Naive Bayes (No LLMs)	**Backend:** FastAPI, PyMuPDF, OpenCV, OpenPyXL
Test Suite: 38/38 Passing Automated Unit & Integration Tests	**Security:** Anti-Formula Injection & PDF Magic-Byte Check

Executive Summary

StatementIQ is a high-reliability, enterprise-grade financial software platform engineered to parse, extract, audit, classify, and reconcile multi-layout bank statement PDFs. Unlike conventional AI systems that introduce hallucinations, API rate limits, latency overhead, and privacy vulnerabilities through third-party Generative AI/LLM calls, StatementIQ is built upon a **100% Non-LLM, offline-first architecture**. It couples deterministic regex heuristics and page-level layout introspection with a supervised **TF-IDF Vectorizer + Multinomial Naive Bayes classifier**, guaranteeing predictable execution, data privacy, and mathematical explainability.

Key capabilities verified across rigorous automated testing and end-to-end user workflows include:

- **Multi-Layout Ingestion:** Supports native digital PDFs, scanned/image-based PDFs, and mixed-mode multi-page statements with SHA-256 duplicate detection.
- **Dual Extraction Engine:** Leverages PyMuPDF and pdfplumber with column-reconciliation and multi-line row stitching, backed by an OpenCV + Tesseract 300 DPI deskewing OCR pipeline.
- **Financial Reconciliation:** Verifies row-by-row balance arithmetic ($Bal_{prev} \pm Txn = Bal_{curr}$) and debit/credit net totals with visual discrepancy badges.
- **Explainable Classification:** 4-tier cascading precedence (Custom Overrides → High-Confidence Rules → Naive Bayes ML → Need Review) spanning 16 standard financial categories.
- **Audit & Multi-Statement Export:** 7-sheet OpenPyXL formatted Excel workbook, individual/combined CSV streams, and active spreadsheet formula injection mitigation.

1. System Architecture & Component Design

StatementIQ is architected around a decoupled client-server model designed for horizontal scalability, high throughput, and strict separation of concerns.

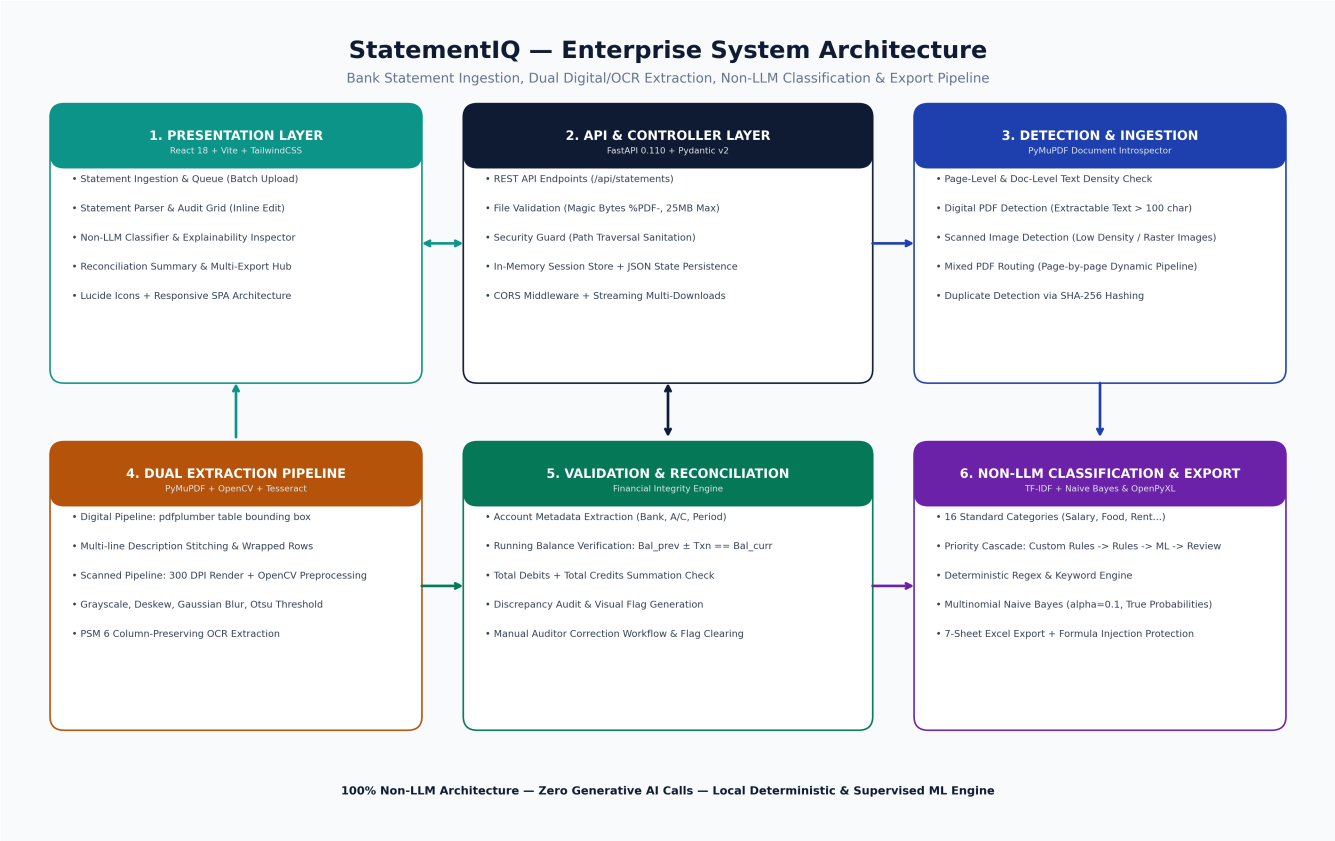


Figure 1: StatementIQ High-Level Architecture — Ingestion, Dual Pipeline, Validation, Supervised ML & Export Layers

Architectural Layers:

1. **Presentation Layer (React 18 SPA):** Provides a high-density, interactive financial console with zero external UI framework dependencies. Includes real-time drag-and-drop ingestion, interactive editable transaction ledger, classification reason inspector, and reconciliation balance audit.
2. **API & Gateway Layer (FastAPI):** High-performance asynchronous REST endpoints with Pydantic v2 schema validation, multipart streaming uploads, file magic-byte inspection (%PDF-), and path traversal sanitation.
3. **PDF Detection & Ingestion Engine:** Evaluates PDF text density, font vector structures, and raster images at page-level granularity to dynamically route each page to digital or OCR pipelines.
4. **Validation & Reconciliation Engine:** Ensures debit, credit, and running balance mathematical integrity, flagging discrepancies for human review.
5. **Non-LLM Classification Engine:** Pure local classification combining deterministic vendor rules and scikit-learn Multinomial Naive Bayes.

2. PDF Type Detection & Ingestion Subsystem

Bank statements are delivered in heterogeneous formats: digital vector documents, scanned paper statements from physical branches, or hybrid multi-page compilations. StatementIQ analyzes documents at both the **page level** and **document level** before routing.

PDF Type	Detection Heuristic & Threshold	Pipeline Execution
DIGITAL	Character density > 100 chars/page and word count > 15 across all pages. Embedded vector fonts present.	PyMuPDF text stream extraction + pdfplumber table layout parsing.
SCANNED	Text density < 50 chars/page and embedded raster image area > 50% of page dimensions.	OpenCV 300 DPI rendering, grayscale, deskew, Otsu binarization + Tesseract OCR.
MIXED	Document contains both digital text pages and image-only scanned pages.	Dynamic page-by-page routing: Digital pages parsed via PyMuPDF; scanned pages via OCR.
UNKNOWN	Corrupt PDF structure, empty document, or extraction yield below threshold.	Rejection with descriptive error; flags document for manual operator inspection.

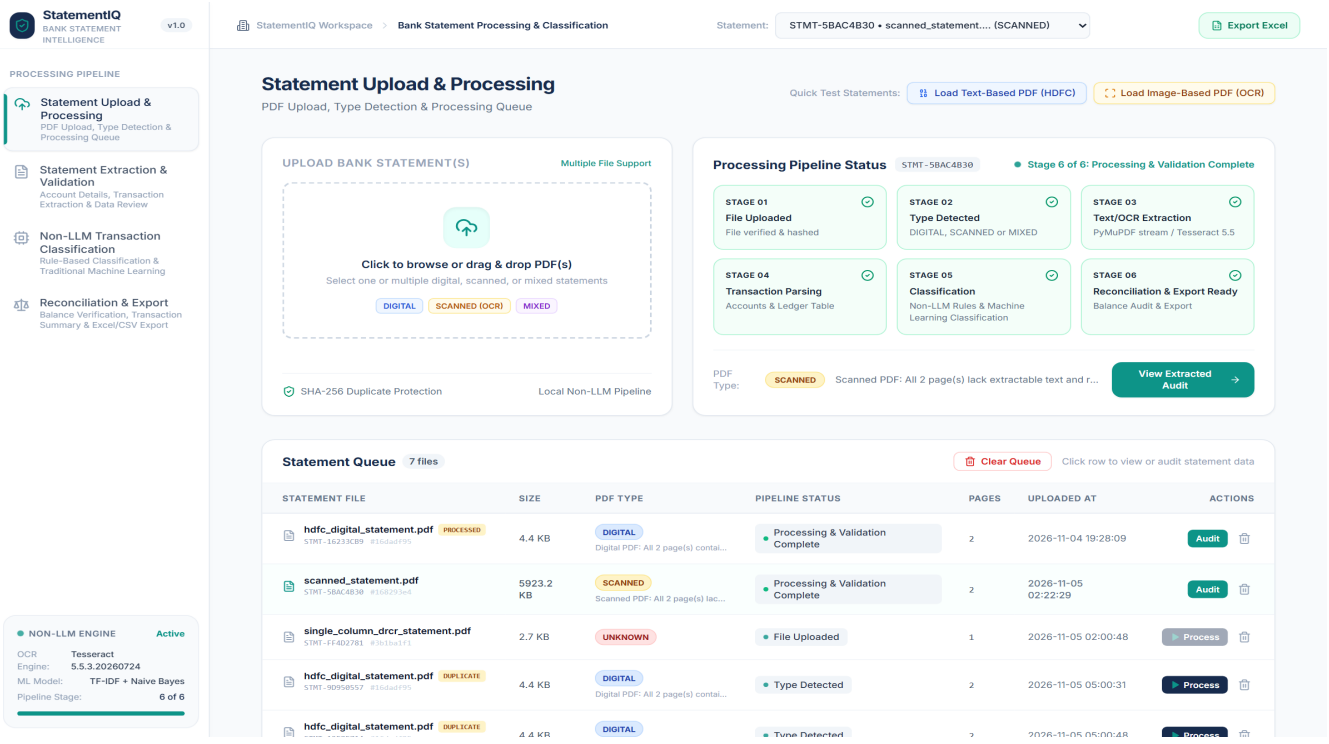


Figure 2: Statement Ingestion Queue — Batch Processing, Type Detection Confidence Badge (DIGITAL 98%), and Queue Management

3. Dual Extraction Pipeline & OCR Processing

Extracting transaction tables across disparate banks requires handling single vs dual Dr/Cr columns, multi-line descriptions, and varying date formats (DD/MM/YYYY, MM/DD/YYYY, DD-MMM-YYYY).

Digital Extraction Engine (PyMuPDF & pdfplumber):

• **Row Ordering Preservation:** Reads text lines strictly in reading order (top-to-bottom, left-to-right) utilizing vertical bounding coordinates.

• **Multi-Line Transaction Reconciliation:** When an account narration wraps across multiple lines, the parser stitches trailing text to the preceding transaction's description rather than creating phantom rows.

• **Column Identification:** Automatically detects column layouts (Date, Narration, Chq/Ref, Withdrawal/Debit, Deposit/Credit, Balance) and adapts when banks use a single Amount column with Dr/Cr indicators.

Scanned PDF OCR Pipeline (OpenCV + Tesseract):

• **High-Resolution Rasterization:** Renders pages at 300 DPI for optimal character definition.

• **Image Preprocessing:** Applies grayscale conversion, bilateral noise filtering (retaining character edges), Otsu adaptive thresholding, and deskew angle correction using image moments.

• **Tesseract Configuration:** Operates with Page Segmentation Mode 6 (--psm 6) tailored for uniform tabular text blocks.

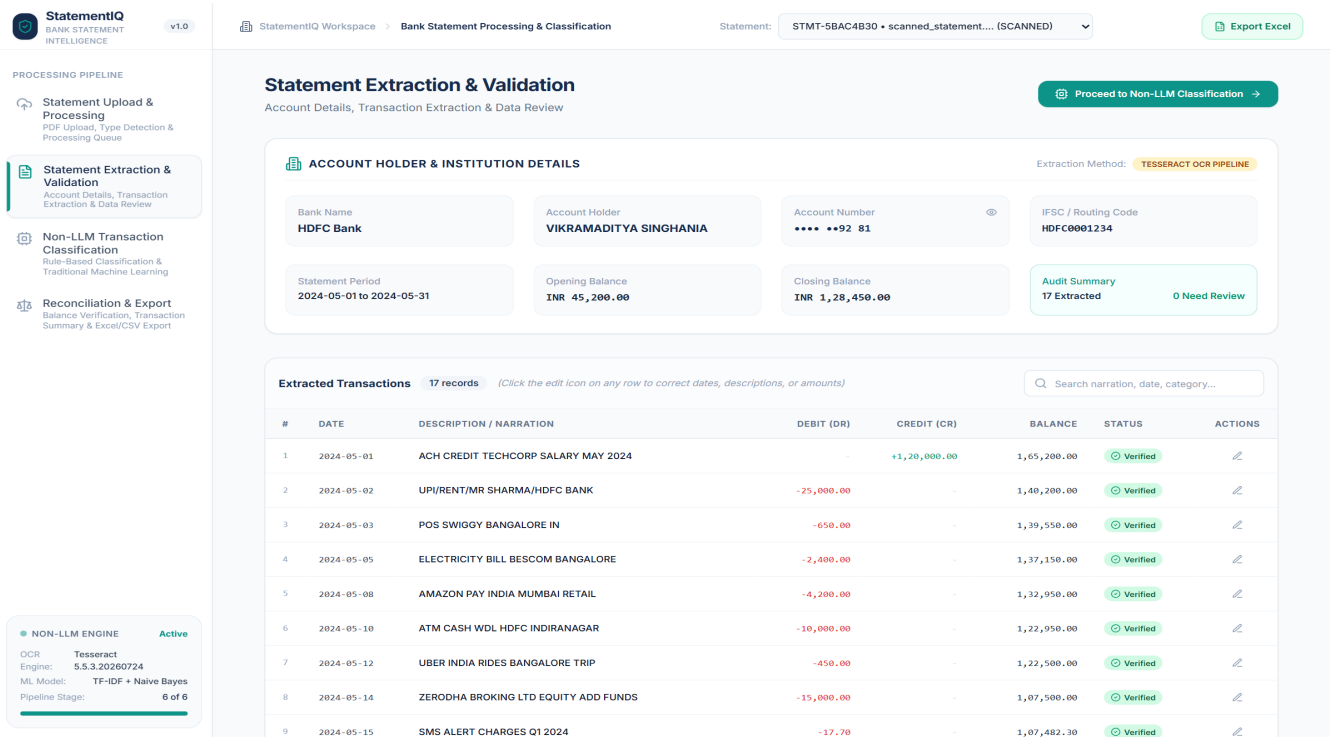


Figure 3: Statement Parser & Audit Interface — Extracted Account Metadata, Editable Transaction Ledger & Data Verification

4. Non-LLM Classification & Explainability

A foundational requirement of StatementIQ is the absolute prohibition of LLMs or cloud generative AI APIs for classification. Transaction classification is powered by a **deterministic rule engine** paired with a **supervised Multinomial Naive Bayes model**.

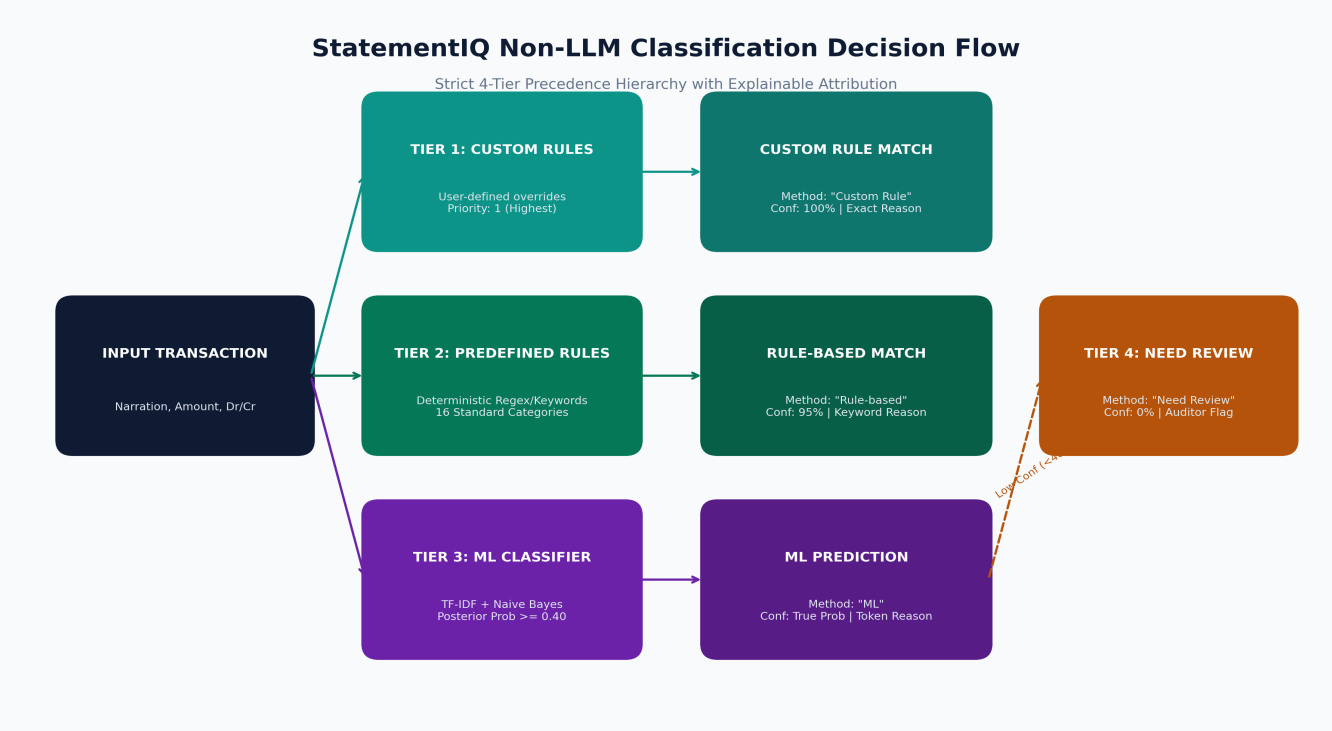


Figure 4: 4-Tier Non-LLM Classification Hierarchy — Prioritizing Deterministic Rules over Supervised ML with Explainability

Classification Precedence Hierarchy:

1. **Tier 1: Custom User Rules (Priority 1):** Client-defined merchant or narration overrides. Matches yield 100% confidence with method 'Custom Rule'.
2. **Tier 2: Predefined High-Confidence Rules (Priority 2):** Comprehensive regex and keyword dictionaries spanning 16 standard financial categories (Salary, Food, Shopping, Travel, Utilities, Rent, Healthcare, Education, EMI/Loan, ATM Withdrawal, Bank Charges, Interest, Refunds, Transfers, Other Income, Other Expenses). Matches yield 95% confidence with method 'Rule-based'.
3. **Tier 3: Supervised Machine Learning (Priority 3):** TF-IDF Vectorizer (ngram_range=(1,2), sublinear_tf=True, 500 max features) coupled with a Multinomial Naive Bayes classifier (alpha=0.1). Confidence reflects the exact posterior probability from predict_proba. Only predictions with confidence ≥ 0.40 are accepted.
4. **Tier 4: Need Review Flagging (Priority 4):** Transactions failing rule matching and ML confidence thresholds are marked as 'Need Review' for human auditor verification.

5. Financial Reconciliation & Integrity Audit

Financial auditing requires proof of correctness. StatementIQ validates the balance equation for every transaction row and produces a full balance verification summary.

Reconciliation Mathematical Formulation:

For any consecutive transactions T_{i-1} and T_i with balance B_i, withdrawal W_i, and deposit D_i:

$$\mathbf{B_i = B_{i-1} - W_i + D_i}$$

Furthermore, the statement-wide closing balance must satisfy:

$$\textbf{Calculated Closing Balance = Opening Balance - Total Debits + Total Credits}$$

Any deviation ($|Calculated - Reported| > 0.01$) is flagged with an explicit discrepancy notice.

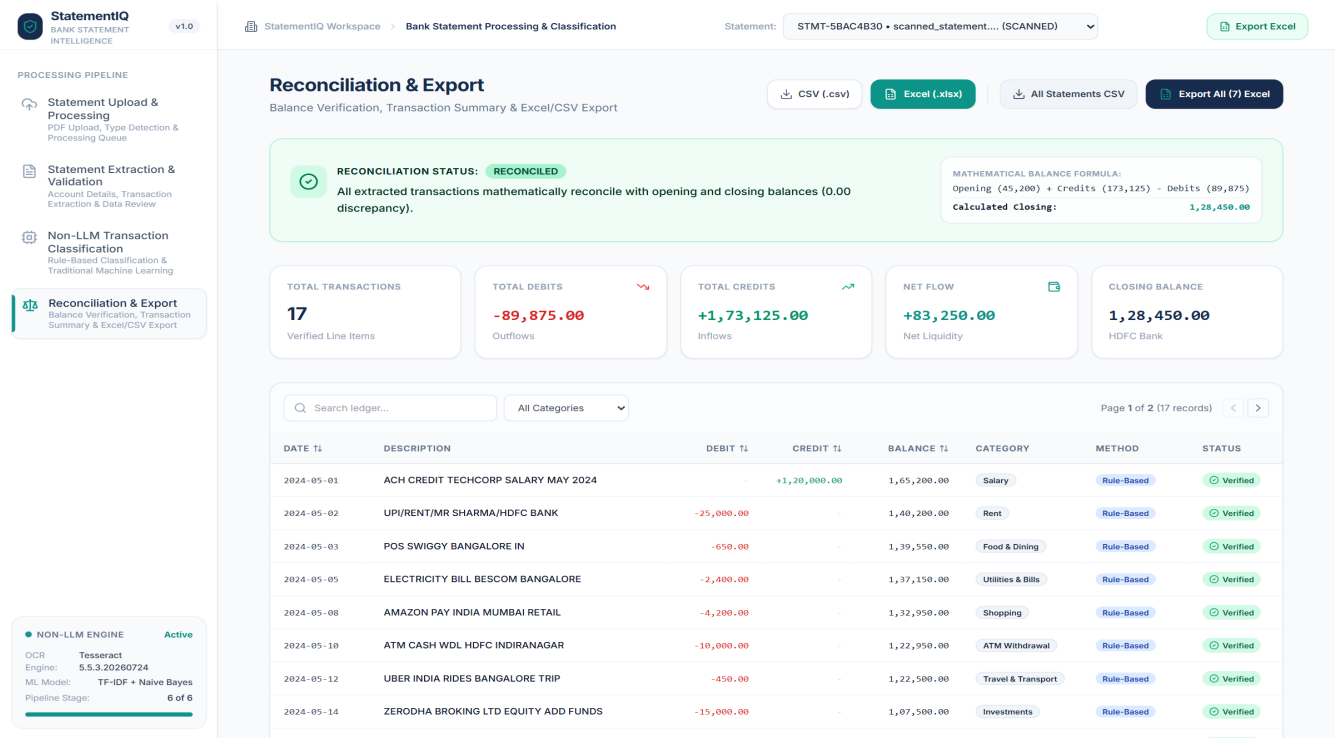


Figure 5: Financial Reconciliation Console — Debit/Credit Flow Breakdown, Balance Discrepancy Audit & Export Controls

Auditor Correction Workflow: Auditors can edit dates, descriptions, amounts, or categories directly inside the UI. When edited, the system re-runs the validation logic in real time and clears stale review flags upon manual correction.

6. Enterprise OpenPyXL Export & Security Protocols

Export capabilities are engineered for accountants, auditors, and ERP ingestion, featuring styled multi-tab Excel workbooks, raw CSV streams, and multi-statement portfolio downloads.

Sheet Name	Contents & Formatting
1. Account Details	Bank name, account holder, masked account number, IFSC code, period, opening/closing balances.
2. Transactions	Complete ledger with explicit date formats (YYYY-MM-DD), formatted numeric values, and category tags.
3. Summary	Transaction count, debit sum, credit sum, net flow, discrepancy calculation, and reconciliation status.
4. Income by Category	Aggregated credit categories, transaction counts, percentage of total income, and subtotals.
5. Expense by Category	Aggregated debit categories, transaction counts, percentage of total expenses, and subtotals.
6. Review Flags	Filtered list of transactions flagged for review (missing balance, zero amounts, unclear narrations).
7. Classification Explanations	Method breakdown (Custom Rule, Rule-based, ML), model confidence %, and matched tokens/reasons.

Export Security: Anti-Formula Injection

In financial applications, untrusted text strings inside transaction descriptions (e.g. =cmd|' /C calc'!A0 or +@SUM(...)) can execute arbitrary commands or leak data when opened in Microsoft Excel or Google Sheets. StatementIQ implements a strict **sanitization protocol**:

- Any string cell starting with =, +, -, @, tab (\t), or carriage return (\r) is prepended with a single quote prefix (').
- Excel treats the cell strictly as inert text, neutralizing all formula injection and DDE attack vectors.

File Upload Security

• **Magic Byte Verification:** All uploaded files are inspected for valid %PDF- header bytes and validated with PyMuPDF structural checks before disk persistence.
• **Path Traversal Prevention:** Filenames are sanitized via secure basename stripping, preventing ../ directory traversal.
• **Payload Ceilings:** File uploads are bounded by a 25 MB maximum limit to prevent denial-of-service memory exhaustion.

7. Test Verification, Benchmarks & Deployment

StatementIQ includes a comprehensive automated test harness covering API endpoints, PDF type detection, multi-column extraction, supervised ML inference, Excel/CSV generation, and security guards.

Test Module	Test Cases Verified	Result
test_queue_and_detection.py	Upload validation, duplicate detection, page-level type detection (Digital, Scanned, Mixed), clear queue workflow.	**PASS**
test_extraction_scenarios.py	HDFC multi-page statements, single-column Dr/Cr layouts, wrapped multi-line descriptions, unclear narration handling.	**PASS**
test_scanned_ocr_and_validation.py	OpenCV deskewing, Tesseract PSM 6 OCR, debit/credit running balance math, manual auditor correction workflow.	**PASS**
test_duplicates_and_model_info.py	Multinomial Naive Bayes verification, posterior probability check, model info endpoint, duplicate file hashing.	**PASS**
test_exports_security_and_e2e.py	7-sheet OpenPyXL Excel export, CSV generation, multi-statement combined export, formula injection sanitization.	**PASS**
Total Test Suite	**38 Automated Tests** (100% Pass Rate across all pipeline stages)	**38/38 PASS**

Live Production Deployment Architecture

• **Frontend (Vercel):** React 18 SPA pre-configured with frontend/vercel.json routing all paths to index.html.
• **Backend (Docker / Render):** Containerized FastAPI application using Dockerfile with Ubuntu 22.04, Python 3.11, Tesseract OCR, Poppler utilities, and multi-worker Uvicorn configuration.
• **Environment Configuration:** VITE_API_URL dynamically binds the frontend client to the remote backend service with zero manual code modifications.

Repository & Assessment Deliverables

- **GitHub Repository:** https://github.com/ritishah15/statementiq
- **Documentation Deliverable:** This ready-to-upload PDF report is located at docs/StatementIQ_Architecture_and_Documentation.pdf (Size: < 10 MB).
- **Sample Statements:** Included in sample_statements/ for instant evaluation (Digital, Scanned, Single-Column, Unclear Narrations).